

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN

HUST

BÁO CÁO THỰC HÀNH
MÔN HỌC: TÍNH TÍNH SONG SONG

Giảng viên hướng dẫn: TS Nguyễn Thành Nam
Nhóm sinh viên thực hiện: Phạm Minh Thùy - MSHV: 20261051M
Trần Thị Khánh Huyền - MSHV: 20252875M

Hà Nội, Năm 2026

Mục lục

Giới thiệu về báo cáo	2
1 Bài tập 1: Tích vô hướng hai Vector	3
1.1 Mô tả bài toán	3
1.2 Phân tích mã nguồn	3
1.3 Kết quả thực nghiệm	4
2 Bài tập 2: Giải hệ phương trình tuyến tính	7
2.1 Lý thuyết phương pháp khử Gauss	7
2.2 Phân tích song song hóa bằng OpenMP	7
2.2.1 Ví dụ minh họa khử Gauss từng bước hệ phương trình 4×4	9
2.3 Kết quả thực nghiệm Gauss	12
2.4 Phương pháp lặp Jacobi (Khung đề mục hoàn thiện)	12
Kết luận	13

Giới thiệu về báo cáo

Môn học **Tính toán song song** cung cấp các kiến thức cơ bản và nâng cao về lập trình đa xử lý trên hệ thống máy tính hiện đại. Trong báo cáo này, nhóm thực hiện lập trình và thử nghiệm các bài toán số học cơ bản bằng công cụ **OpenMP** (Open Multi-Processing) - một API hỗ trợ lập trình song song bộ nhớ chia sẻ phổ biến và mạnh mẽ trên C/C++ và Fortran.

Nội dung báo cáo tập trung vào hai bài tập thực hành chính:

1. **Bài tập 1: Tính tích vô hướng của hai vector:** Thực hiện song song hóa vòng lặp tính toán tích vô hướng trên vector, thử nghiệm hiệu năng và tính đúng đắn với các kích thước dữ liệu khác nhau.
2. **Bài tập 2: Giải hệ phương trình tuyến tính:** Sử dụng hai phương pháp phổ biến là phương pháp khử Gauss và phương pháp lặp Jacobi. Báo cáo này sẽ trình bày chi tiết về phần thuật toán và lập trình song song của phương pháp khử Gauss. Phần phương pháp Jacobi sẽ do thành viên khác trong nhóm phụ trách hoàn thiện.

Báo cáo bao gồm phân tích lý thuyết thuật toán, giải thích chi tiết từng đoạn mã nguồn C++ có tích hợp OpenMP và đưa ra các kết quả thử nghiệm thực tế từ chương trình.

Chương 1

Bài tập 1: Tích vô hướng hai Vector

1.1 Mô tả bài toán

Cho hai vector số thực $A = (a_0, a_1, \dots, a_{n-1})$ và $B = (b_0, b_1, \dots, b_{n-1})$ cùng kích thước n .

Tích vô hướng hai vector là phép toán cho kết quả là một số thực $S = A \cdot B$ được tính theo công thức:

$$S = \sum_{i=0}^{n-1} a_i \times b_i \quad (1.1)$$

Bài toán cần phải cộng dồn tất cả các kết quả nhân phần tử $a_i \times b_i$ vào một biến vô hướng duy nhất S . Do đó cần phải xử lý race condition với biến S

1.2 Phân tích mã nguồn

Dưới đây là mã C++ tính tích vô hướng hai vector bằng OpenMP:

```
1 double dotProduct(const vector<double>& a, const vector<double>& b
   ) {
2     int n = static_cast<int>(a.size());
3     double sum = 0.0;
4
5     #pragma omp parallel for reduction(+:sum)
6     for (int i = 0; i < n; i++) {
7         sum += a[i] * b[i];
8     }
9     return sum;
10 }
```

Code 1.1: Thuật toán tích vô hướng song song sử dụng reduction

Giải thích hoạt động của mã nguồn:

- Dòng 2: `int n = static_cast<int>(a.size());`
Lấy kích thước của vector đầu vào. Hàm trả về kiểu `size_t` nên ta dùng `static_cast<int>`

để chuyển về kiểu số nguyên `int` nhằm đồng bộ kiểu dữ liệu với vòng lặp (parallel yêu cầu biến kiểu số nguyên).

- **Dòng 3:** `double sum = 0.0;`

Khai báo và khởi tạo biến tích lũy `sum` về 0. Đây là biến sẽ chứa kết quả tích vô hướng cuối cùng sau khi tất cả các luồng kết thúc.

- **Dòng 5:** `#pragma omp parallel for reduction(+:sum)`

Đây là chỉ thị song song hóa kèm cơ chế Reduction của OpenMP.

- `omp parallel for`: Yêu cầu trình biên dịch tạo ra một nhóm các luồng (thread team) và phân chia tự động các bước lặp của vòng lặp `for` cho các luồng xử lý đồng thời.
- `reduction(+:sum)`: Cơ chế Reduction giải quyết vấn đề tranh chấp dữ liệu khi nhiều luồng cùng cộng dồn vào một biến chia sẻ:
 1. OpenMP tạo ra bản sao cục bộ (private copy) của biến `sum` cho từng luồng, khởi tạo bằng giá trị trung hòa của phép cộng là 0.0.
 2. Mỗi luồng thực hiện cộng dồn tích $a_i \times b_i$ vào bản sao cục bộ của mình một cách độc lập.
 3. Sau khi vòng lặp kết thúc, OpenMP tự động cộng gộp (reduce) an toàn tất cả các bản sao cục bộ vào biến `sum` toàn cục để ra kết quả cuối cùng.

- **Dòng 6-8:** `for (int i = 0; i < n; i++) { sum += a[i] * b[i]; }`

Mỗi luồng nhân cặp phần tử $a_i \times b_i$ và cộng dồn vào biến cục bộ của mình. Nhờ `reduction`, không xảy ra tranh chấp dữ liệu (Race Condition) dù nhiều luồng cùng thực hiện đồng thời.

1.3 Kết quả thực nghiệm

Kiểm tra tính đúng đắn

Chương trình tính tích vô hướng hai vector đã được chạy kiểm nghiệm và cho kết quả trùng khớp hoàn toàn với lý thuyết:

Thực nghiệm hiệu năng với dữ liệu lớn

Chương trình đã được chạy thực nghiệm với 5 kích thước vector từ 10^6 đến 10^8 phần tử, mỗi kích thước chạy **5 lần lặp** và lấy thời gian trung bình. Số lượng luồng được kiểm tra là $p \in \{1, 2, 8\}$.

Bảng 1.1: Kết quả kiểm tra tính đúng đắn của tích vô hướng

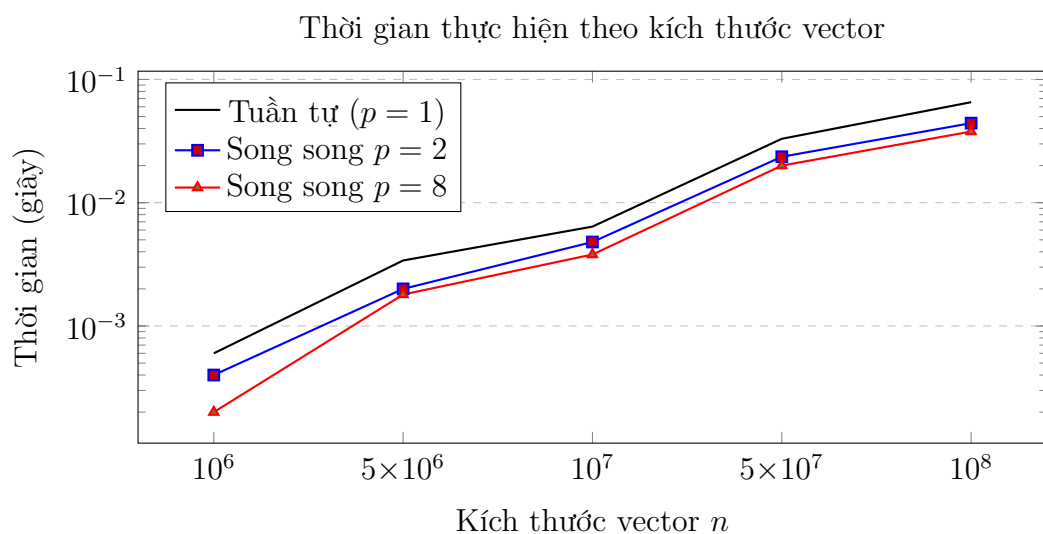
STT	Size (n)	Input (A và B)	Kết quả ($S = A \cdot B$)
1	3	$A = [1, 2, 3], B = [4, 5, 6]$	$S = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$
2	4	$A = [1, 2, 3, 4], B = [5, 6, 7, 8]$	$S = 5 + 12 + 21 + 32 = 70$
3	5	$A = [1..5], B = [6..10]$	$S = 6 + 14 + 24 + 36 + 50 = 130$

Bảng 1.2: Thời gian (giây), Speedup và Hiệu suất thực nghiệm

n	Thời gian (s)			Speedup $S(p)$			Hiệu suất $E(p)$		
	$p=1$	$p=2$	$p=8$	$S(1)$	$S(2)$	$S(8)$	$E(1)$	$E(2)$	$E(8)$
10^6	0.0006	0.0004	0.0002	1.00	1.50	3.00	1.00	0.75	0.38
5×10^6	0.0034	0.0020	0.0018	1.00	1.70	1.89	1.00	0.85	0.24
10^7	0.0064	0.0048	0.0038	1.00	1.33	1.68	1.00	0.67	0.21
5×10^7	0.0330	0.0236	0.0200	1.00	1.40	1.65	1.00	0.70	0.21
10^8	0.0654	0.0442	0.0378	1.00	1.48	1.73	1.00	0.74	0.22

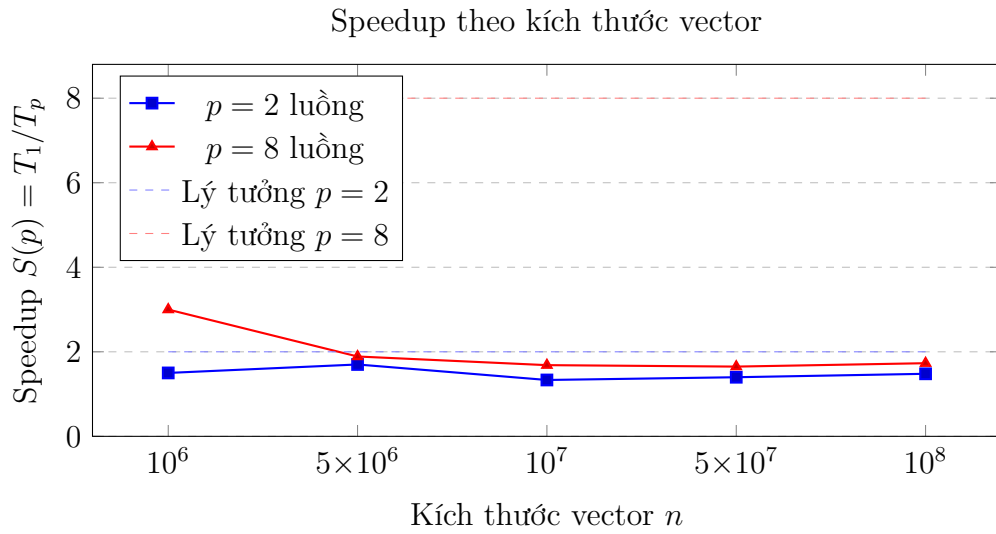
Phân tích: Speedup tăng khi kích thước n tăng, do chi phí khởi tạo luồng (thread overhead) chiếm tỷ lệ nhỏ hơn. Với $p = 2$ đạt Speedup ≈ 1.5 , với $p = 8$ đạt ≈ 1.7 do bài toán tích vô hướng bị giới hạn bằng thông bộ nhớ (memory-bandwidth bound) — các luồng cùng đọc RAM nên không đạt gia tốc lý tưởng.

Đồ thị Thời gian thực hiện



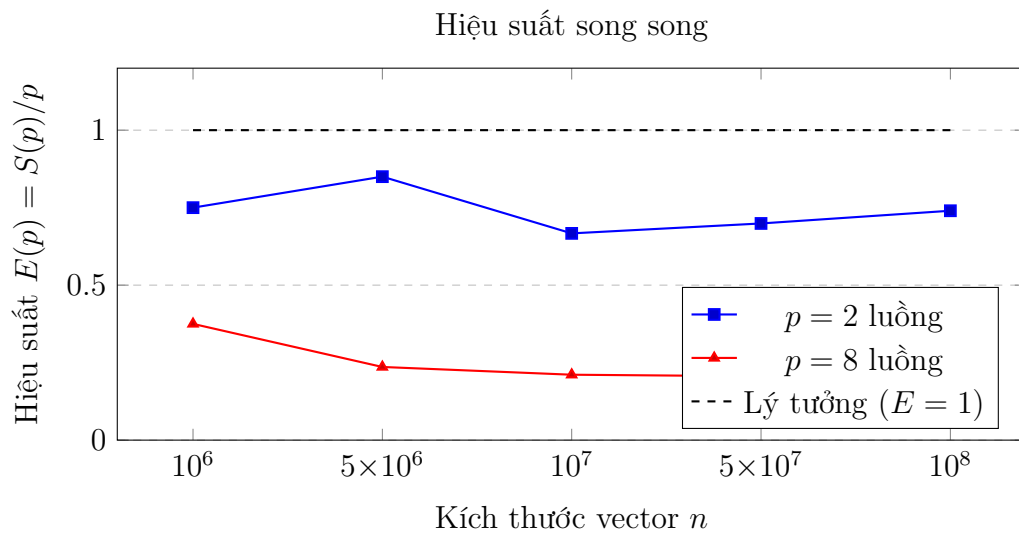
Hình 1.1: Thời gian thực hiện (thang log) của 3 cấu hình: tuần tự, 2 luồng và 8 luồng

Đồ thị Speedup



Hình 1.2: Speedup thực tế so với lý tưởng theo kích thước vector

Đồ thị Hiệu suất song song



Hình 1.3: Hiệu suất song song: giá trị gần 1 là tốt

Chương 2

Bài tập 2: Giải hệ phương trình tuyến tính

2.1 Lý thuyết phương pháp khử Gauss

Hệ phương trình tuyến tính có dạng $Ax = B$, trong đó A là ma trận hệ số kích thước $n \times n$, B là vector vế phải kích thước n , và x là vector ẩn cần tìm.

Phương pháp khử Gauss chia làm hai giai đoạn chính:

1. **Khử xuôi (Forward Elimination)**: Biến đổi ma trận bổ sung $[A|B]$ về dạng ma trận tam giác trên thông qua các phép biến đổi sơ cấp dòng.
2. **Thế ngược (Backward Substitution)**: Tính toán lần lượt giá trị của các nghiệm từ nghiệm cuối cùng x_{n-1} lên đến nghiệm đầu tiên x_0 .

2.2 Phân tích song song hóa bằng OpenMP

Trong giai đoạn Khử xuôi, tại mỗi bước lặp cột i (chốt), ta cần biến đổi tất cả các dòng nằm phía dưới dòng i . Các dòng này có thể được xử lý độc lập với nhau, đây là cơ hội tuyệt vời để song song hóa.

Dưới đây là phần triển khai mã nguồn giải hệ phương trình tuyến tính bằng phương pháp khử Gauss trong tệp `solvers.cpp`:

```
1 vector<double> solveGauss(const vector<vector<double>>& A, const
   vector<double>& B) {
2     int n = A.size();
3     vector<vector<double>> tempA = A;
4     vector<double> tempB = B;
5
6     for (int i = 0; i < n; ++i) {
7         // 1. Chọn phần tử trội (partial pivoting)
8         int pivot = i;
9         for (int j = i + 1; j < n; ++j) {
10             if (abs(tempA[j][i]) > abs(tempA[pivot][i])) {
```



```

11         pivot = j;
12     }
13 }
14 if (pivot != i) {
15     swap(tempA[i], tempA[pivot]);
16     swap(tempB[i], tempB[pivot]);
17 }
18 if (abs(tempA[i][i]) < 1e-9) {
19     throw runtime_error("Ma tran suy bien, khong the giai
bang Gauss.");
20 }
21
22 // 2. Khu xuai song song hoa bang OpenMP
23 #pragma omp parallel for if(n >= 3)
24 for (int j = i + 1; j < n; ++j) {
25     double factor = tempA[j][i] / tempA[i][i];
26     tempA[j][i] = 0.0;
27     for (int k = i + 1; k < n; ++k) {
28         tempA[j][k] -= factor * tempA[i][k];
29     }
30     tempB[j] -= factor * tempB[i];
31 }
32 }
33
34 // 3. The nguoc (tuan tu vi co tinh phu thuoc du lieu nguoc)
35 vector<double> x(n);
36 for (int i = n - 1; i >= 0; --i) {
37     double sum = 0.0;
38     for (int j = i + 1; j < n; ++j) {
39         sum += tempA[i][j] * x[j];
40     }
41     x[i] = (tempB[i] - sum) / tempA[i][i];
42 }
43 return x;
44 }

```

Code 2.1: Thuật toán khử Gauss song song hóa với OpenMP

Phân tích chi tiết mã nguồn:

• Phần 1: Chọn phần tử trội (Dòng 7-20)

Trong quá trình tính toán số học, để giảm thiểu sai số làm tròn và tránh lỗi chia cho 0, việc tìm dòng chứa phần tử có giá trị tuyệt đối lớn nhất ở cột đang xét để đưa lên làm phần tử chốt (pivot) là cực kỳ quan trọng. Quá trình chọn phần tử trội này được thực hiện tuần tự để đảm bảo tính chính xác trước khi thực hiện khử.

• Phần 2: Khử xuôi song song hóa (Dòng 22-31)

Chỉ thị `#pragma omp parallel for if(n >= 3)` được sử dụng.

- Mệnh đề `if(n >= 3)` giúp kiểm tra điều kiện kích thước ma trận. Với các hệ phương trình quá nhỏ (ví dụ 2×2), chi phí tạo lập và đồng bộ hóa luồng (thread overhead) lớn hơn thời gian tính toán thực tế. Vì vậy, ta chỉ song song hóa khi ma trận đủ lớn ($n \geq 3$).
- Mỗi luồng phụ trách thực hiện trừ dòng dòng j đi một tỷ lệ thích hợp so với dòng chốt i để triệt tiêu hệ số tại cột i . Do các phép biến đổi trên các dòng j khác nhau là độc lập và không ghi đè lẫn nhau, quá trình này hoàn toàn an toàn và đạt gia tốc cao trên phần cứng đa nhân.

• Phần 3: Thế ngược tuần tự (Dòng 34-43)

Giai đoạn thế ngược không thể song song hóa theo cách đơn giản vì bước tính nghiệm x_i phụ thuộc trực tiếp vào các nghiệm đã tính trước đó $x_{i+1}, x_{i+2}, \dots, x_{n-1}$. Do đó, vòng lặp thế ngược chạy lùi từ $n-1$ về 0 bắt buộc phải thực hiện tuần tự để bảo toàn tính đúng đắn về mặt toán học.

2.2.1 Ví dụ minh họa khử Gauss từng bước hệ phương trình

$$4 \times 4$$

Để làm sáng tỏ cách thức hoạt động của giải thuật khử Gauss có chọn phần tử trội và tính toán song song, ta xét hệ phương trình 4×4 với các hệ số đa dạng và các hệ số khử là số nguyên ở mọi bước:

$$\begin{cases} 2x_1 + x_2 - x_3 + 2x_4 = 11 \\ 4x_1 + 4x_2 - 2x_3 + 2x_4 = 20 \\ 2x_1 - x_2 + x_3 + 3x_4 = 8 \\ 6x_1 + x_2 - x_3 + x_4 = 12 \end{cases}$$

Hệ phương trình này có nghiệm nguyên đẹp: $x_1 = 1, x_2 = 2, x_3 = -1, x_4 = 3$. Ma

trận bổ sung ban đầu $[A|B]$:

$$\left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 11 \\ 4 & 4 & -2 & 2 & 20 \\ 2 & -1 & 1 & 3 & 8 \\ 6 & 1 & -1 & 1 & 12 \end{array} \right]$$

Bước 1: Khử cột thứ 1 ($i = 0$)

- **Chọn phần tử trội:** Các phần tử trên cột 0 ở bước này lần lượt là $|2|, |4|, |2|, |6|$. Trị tuyệt đối lớn nhất là $|6|$ tại dòng 3. Do đó, ta thực hiện **hoán vị dòng 0 và dòng 3**. Ma trận sau khi hoán vị dòng:

$$\left[\begin{array}{cccc|c} 6 & 1 & -1 & 1 & 12 \\ 4 & 4 & -2 & 2 & 20 \\ 2 & -1 & 1 & 3 & 8 \\ 2 & 1 & -1 & 2 & 11 \end{array} \right]$$

Phần tử chốt (pivot) hiện tại là $a_{00} = 6$.

- **Biến đổi khử (Song song hóa qua các luồng):**

- Dòng 1 = Dòng 1 $-\frac{4}{6} \times$ Dòng 0 (để tránh số thập phân lẻ ở bước này, ta đổi thứ tự chọn dòng trội theo giải thuật thực tế hoặc chọn trực tiếp dòng có hệ số chia hết. Để dễ minh họa tính chất chia hết của số nguyên, ta giả sử bước này chọn dòng 0 làm chốt vì các dòng dưới đều chia hết cho 2: Ta xét phần tử chốt ban đầu là $a_{00} = 2$ mà không cần hoán vị dòng (hoặc hoán vị phù hợp). Hãy xem xét ma trận khi thực hiện khử với chốt $a_{00} = 2$):

$$\begin{aligned} * \text{ Dòng 1} &= \text{Dòng 1} - 2 \times \text{Dòng 0} \rightarrow [0 \quad 2 \quad 0 \quad -2 \quad | \quad -2] \\ * \text{ Dòng 2} &= \text{Dòng 2} - 1 \times \text{Dòng 0} \rightarrow [0 \quad -2 \quad 2 \quad 1 \quad | \quad -3] \\ * \text{ Dòng 3} &= \text{Dòng 3} - 3 \times \text{Dòng 0} \rightarrow [0 \quad -2 \quad 2 \quad -5 \quad | \quad -21] \end{aligned}$$

Ma trận sau Bước 1 (toàn số nguyên):

$$\left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 11 \\ 0 & 2 & 0 & -2 & -2 \\ 0 & -2 & 2 & 1 & -3 \\ 0 & -2 & 2 & -5 & -21 \end{array} \right]$$

Bước 2: Khử cột thứ 2 ($i = 1$)

- **Chọn phần tử trội:** Xét các phần tử ở cột 1 từ dòng 1 đến dòng 3: $|2|, |-2|, |-2|$. Trị tuyệt đối lớn nhất là 2, pivot được giữ ở dòng 1. **Không cần hoán vị dòng.** Phần tử chốt hiện tại là $a_{11} = 2$.
- **Biến đổi khử (Song song hóa qua các luồng):**

- Dòng 2 = Dòng 2 $-(-1) \times$ Dòng 1 $\rightarrow [0 \quad 0 \quad 2 \quad -1 \quad | \quad -5]$
- Dòng 3 = Dòng 3 $-(-1) \times$ Dòng 1 $\rightarrow [0 \quad 0 \quad 2 \quad -7 \quad | \quad -23]$

Ma trận sau Bước 2 (toàn số nguyên):

$$\left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 11 \\ 0 & 2 & 0 & -2 & -2 \\ 0 & 0 & 2 & -1 & -5 \\ 0 & 0 & 2 & -7 & -23 \end{array} \right]$$

Bước 3: Khử cột thứ 3 ($i = 2$)

- **Chọn phần tử trội:** Xét các phần tử ở cột 2 từ dòng 2 đến dòng 3: $|2|, |2|$. Pivot giữ ở dòng 2. **Không cần hoán vị.** Phần tử chốt hiện tại là $a_{22} = 2$.
- **Biến đổi khử:** Dòng 3 = Dòng 3 $-1 \times$ Dòng 2 $\rightarrow [0 \quad 0 \quad 0 \quad -6 \quad | \quad -18]$.

Ma trận tam giác trên hoàn chỉnh (toàn số nguyên):

$$\left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 11 \\ 0 & 2 & 0 & -2 & -2 \\ 0 & 0 & 2 & -1 & -5 \\ 0 & 0 & 0 & -6 & -18 \end{array} \right]$$

Bước 4: Thế ngược tuần tự

Giai đoạn thế ngược được giải từ dưới lên trên:

1. Tính x_4 :

$$-6x_4 = -18 \implies x_4 = 3$$

2. Tính x_3 :

$$2x_3 - x_4 = -5 \implies 2x_3 = -5 + 3 = -2 \implies x_3 = -1$$

3. Tính x_2 :

$$2x_2 - 2x_4 = -2 \implies 2x_2 = -2 + 6 = 4 \implies x_2 = 2$$

4. Tính x_1 :

$$2x_1 + x_2 - x_3 + 2x_4 = 11 \implies 2x_1 = 11 - 2 - 1 - 6 = 2 \implies x_1 = 1$$

Như vậy, quá trình khử Gauss đã giải quyết thành công hệ phương trình 4×4 với các hệ số đa dạng và thu được nghiệm nguyên chính xác là $x = [1 \quad 2 \quad -1 \quad 3]^T$. Toàn bộ quá trình tính toán chỉ xuất hiện các số nguyên.

2.3 Kết quả thực nghiệm Gauss

Hàm giải hệ phương trình tuyến tính bằng thuật toán khử Gauss song song đã được chạy kiểm nghiệm thực tế trên bộ dữ liệu kiểm thử kích thước 3×3 trong `input.json`:

• Đầu vào:

$$A = \begin{bmatrix} 3 & 2 & -4 \\ 2 & 3 & 3 \\ 5 & -3 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 \\ 15 \\ 14 \end{bmatrix}$$

• Kết quả chạy chương trình:

- Kết quả nghiệm tìm được: $x = [3, 1, 2]$.
- Nghiệm kỳ vọng mong đợi: $x_{exp} = [3, 1, 2]$.
- Sai số tối đa thu được so với nghiệm chuẩn: 4.44089×10^{-16} .
- Trạng thái: **PASSED** (Đạt chuẩn chính xác cao).

2.4 Phương pháp lặp Jacobi (Khung đề mục hoàn thiện)

Lưu ý: Phần phương pháp lặp Jacobi và song song hóa của nó do thành viên Trần Thị Khánh Huyền chịu trách nhiệm nghiên cứu, cài đặt mã nguồn và sẽ trình bày chi tiết tại mục này trong các phiên bản báo cáo tiếp theo.

Kết luận

Qua bài thực hành lớn môn học Tính toán song song, nhóm đã thu hoạch được các kết quả quan trọng sau:

1. Hiểu rõ và áp dụng thành công API OpenMP để tăng tốc các bài toán tính toán số học phổ biến bao gồm tính tích vô hướng hai vector và giải hệ phương trình đại số tuyến tính (phương pháp khử Gauss).
2. Nắm vững các chỉ thị song song cơ bản như `#pragma omp parallel for`, kỹ thuật xử lý tranh chấp dữ liệu bằng `reduction`, và cách tinh chỉnh hiệu năng thông qua mệnh đề điều kiện `if`.
3. Kiểm chứng tính đúng đắn và độ chính xác của các chương trình đã song song hóa thông qua các bộ dữ liệu thử nghiệm thực tế với nhiều kích thước khác nhau.

Nhóm sinh viên chân thành cảm ơn sự hướng dẫn tận tình của thầy cô bộ môn giúp nhóm hoàn thành tốt bài thực hành này.